

Design and Practical Implementation of a High Efficiency Two-Layer Trajectory Planning Method for AGV

Runda Zhang¹, Runqi Chai¹, Member, IEEE, Senchun Chai¹, Senior Member, IEEE, Yuanqing Xia², Senior Member, IEEE, and Antonios Tsourdos³, Member, IEEE

Abstract—This article proposes a two-layer trajectory optimization method for the autonomous ground vehicle. This two-layer strategy includes an efficient path planning layer and a fast trajectory planning layer. In the first layer, a novel target area adaptive rapidly exploring random tree algorithm (TAA-RRT*) is proposed to search the shortest path. This layer mainly includes a preprocessing and a sampling planning process. In the preprocessing process, the generalized voronoi diagram is used to construct the environment information and find the initial path. Then, the sampled target area (TA) is constructed based on this initial path to provide nonuniform sampling. In the sampling planning process, the improved adaptive RRT* algorithm is used to carry out sampling planning in the TA, and the direct connection strategy is combined to quickly locate the optimal solution. In the trajectory planning layer, combined with the constraints of the unmanned vehicle and the path constraints obtained in the first layer, the speed planning and the trajectory optimization are addressed by solving the optimal control problem. After performing a large number of experiments, the feasibility and effectiveness of the proposed method are verified.

Index Terms—Autonomous ground vehicle, generalized voronoi diagram, path planning, rapidly exploring random tree algorithm (RRT*), target area, trajectory optimization.

I. INTRODUCTION

AUTONOMOUS ground vehicles have been widely used in various fields of our lives [1], [2], [3], [4]. As an important branch of robotics, motion planning has received extensive attention. Its purpose is to find a collision-free driving path for the robot from the starting point to the end point [5], [6], [7]. Generally speaking, motion planning can be divided into global path planning, local path planning, and hybrid path planning [8].

Manuscript received 23 October 2022; revised 26 January 2023; accepted 16 February 2023. Date of publication 6 March 2023; date of current version 16 August 2023. (Corresponding author: Runqi Chai.)

Runda Zhang, Runqi Chai, Senchun Chai, and Yuanqing Xia are with the School of Automation, Beijing Institute of Technology, Beijing 100081, China (e-mail: runda.zhang@bit.edu.cn; r.chai@bit.edu.cn; chaic97@bit.edu.cn; xia_yuanqing@bit.edu.cn).

Antonios Tsourdos is with the School of Aerospace, Transport and Manufacturing, Cranfield University, Bedford MK43 0AL, U.K. (e-mail: a.tsourdos@cranfield.ac.uk).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TIE.2023.3250847>.

Digital Object Identifier 10.1109/TIE.2023.3250847

Global path planning's purpose is to plan a feasible path for the robot under the condition of known static obstacles [9]. At present, the existing global path planning algorithms can be roughly divided into three types: the algorithm based on graph theory, the algorithm based on sampling, and the algorithm based on swarm intelligence [10], [11], [12]. Among them, sampling-based algorithms, such as the probabilistic roadmap (PRM) [13] and the rapidly exploring random tree (RRT) [14], are gaining popularity due to their ability to solve high-dimensional problems. These algorithms have also been shown to guarantee probabilistic completeness and asymptotic optimality [15].

However, an important problem faced by sampling-based algorithms is that they have large computational requirements. Therefore, more and more people are committed to improving the efficiency of the sampling algorithm to speed up the convergence speed of the algorithm. Due to the probabilistic and blind nature of sampling algorithms, many researchers have adopted various methods to improve the performance of sampling algorithms [16]. Nonuniform sampling is one way to improve the performance of sampling algorithms. In [17], a nonuniform sampling strategy is proposed, after finding the initial path, the subsequent sampling is concentrated in the ellipsoid region to improve the convergence speed. Zhang et al. [18] used a Gaussian mixture regression approach to learn key features from human demonstrations to obtain the probability distribution of human trajectories and provide nonuniform sampling. In [19], the author synthesized the different characteristics of different sampling algorithms, used hybrid incremental search and dynamic optimization search to quickly find feasible solutions. Although the above methods can improve the speed of convergence, the quality of the obtained paths is not guaranteed, so they are not practical.

At the same time, path quality is also a particularly important aspect [20]. Salzman et al. [21] proposed a lower bound tree RRT, which introduced an approximation factor to determine the quality of the path. Wang proposed the EB-RRT in [22], which utilizes the elastic band (EB) method to continuously shorten the path after generating the initial path to improve the path quality. In [23], it was found that the step size of sampling has a certain influence on the time and quality of path finding. A larger step size is beneficial to the exploration of the region, and a smaller step size is beneficial to the improvement of path details. This idea of variable step size is also used in [24]. When approaching

the obstacle, the variable expansion step size is used to move away from the trap area. Even if the initial path is found, the AGV cannot directly follow the path planned by the sampling algorithm due to the kinematic and dynamic constraints of the AGV [25], which complicates the problem.

For the path smoothing problem, some existing local path planning methods can be effectively handled, such as the dynamic window approach (DWA) [26] and the time elastic band (TEB) [27]. Unfortunately, DWA cannot avoid some special obstacles very well, and TEB is prone to large fluctuations in the calculation of velocity acceleration, resulting in inaccuracy. The paths calculated by these methods are not the global optimal path. There are also many scholars who use the function curve method to smooth the path [28], but it is conceivable that when the number of control points increases, the amount of calculation will also increase sharply. Recently, optimization-based methods stand out for their accuracy, which models the entire trajectory planning problem as an optimal control problem for solution [29], [30]. However, this method often requires a large amount of computation. In [31], the author proposed a two-stage method, in which the initial guess is obtained through the first stage and then the optimization solution in the second stage is carried out, but we found that the solution speed still needs to be improved.

Importantly, GVD methods are widely used in path-based planning problems [32]. Processing maps with GVD methods can obtain routes that maximize the distance to obstacles [33], [34]. So we can analyze the static information of the scene map by drawing the GVD map.

Combining this information, this article proposes a two-layer trajectory planning method for AGVs. First, the first layer uses the sampling algorithm to plan the shortest path, and then the second layer combines the path obtained by the first layer to perform speed planning and trajectory optimization. In the first layer, we propose the TAA-RRT* algorithm for path planning. At the same time, according to the environmental information of the TA, an appropriate step size is adaptively selected during sampling. Then, the direct connection strategy is adopted, which not only further shortens the path, but also obtains accurate path constraints and corner constraints, and it transforms the trajectory planning problem of the second layer into an OCP. It is worth noting that in the second layer our path constraints also reduce the solution space of the whole problem, and do not consider obstacle constraints, so this will be a lightweight process, further speeding up the solution. Finally, the optimal path for the AGV to travel is obtained by solving the OCP by using the Gaussian pseudospectral method.

The main contributions of this article are as follows:

- 1) Based on the two-layer method, we use the designed planning method to find the optimal path, and then use the optimization-based method to perform high-quality optimization.
- 2) In the path planning layer, TAA-RRT* is an efficient path planning algorithm based on nonuniform sampling technology proposed by us. By establishing the target area and storing the environment information, we concentrate the sampling in the target area, and adaptively set the

expansion step according to the environment information during sampling. Finally, the direct connection strategy is adopted to further optimize the path. While sampling efficiently, TAA-RRT* can adapt to different environments and quickly converge to a stable path solution.

- 3) We analyzed three theoretical characteristics of TAA-RRT*, and compared TAA-RRT* and RRT* from the theoretical level.
- 4) At the optimization layer, we establish collision-free path constraints based on the first-layer path solutions. Combined with vehicle kinematic constraints, we optimize the path and perform velocity planning by building and solving the OCP. Finally, a high-quality path solution is obtained.

The rest of this article is organized as follows. Section II presents the problem statements of path planning. In Section III, the detailed flow of the TAA-RRT* algorithm is introduced. Section IV discusses the constraints of the AGV and the OCP. In Section V, a large number of experiments are carried out to verify the feasibility of the method. Finally, Section VI concludes this article.

II. PROBLEM FORMULATION

In this section, we describe the path planning problem in technical terms, review the RRT* algorithm, and discuss its limitations.

A. Definitions

To formulate the path planning problem clearly, we abstract the working environment into a configuration space. Let $X \subset R^n$ be the configuration space. Let X_{obs} be the set of obstacles. The free space is represented as $X_{\text{free}} = X \setminus X_{\text{obs}}$, which is the set of feasible states. The purpose of the first layer is to find a feasible path $\sigma(t)$ such that

$$\sigma : [0, T] \longrightarrow X_{\text{free}}, \sigma(0) = x_{\text{init}}, \sigma(T) = x_{\text{goal}} \quad (1)$$

where x_{init} is the initial state, x_{goal} is the goal state, and t is the time stamp. Let Σ represents the set of all feasible paths. The cost function $\text{Cost}(\sigma)$ represents the cost of a feasible path from x_{init} to x_{goal} according to the Euclidean distance. The optimal path planning problem can be defined as

$$\begin{aligned} \sigma^* &= \arg \min_{\sigma \in \Sigma} \text{Cost}(\sigma) \\ \text{s.t. } \sigma(0) &= x_{\text{init}} \\ \sigma(T) &= x_{\text{goal}} \\ \sigma(t) &\in X_{\text{free}}(t) \quad \forall t \in [0, T]. \end{aligned} \quad (2)$$

B. Limitations of RRT*

The RRT* algorithm is an incremental sampling algorithm, it has been proven that the path found by the RRT* algorithm is asymptotically optimal [15]. Although the RRT* algorithm has good performance, it also has the following limitations.

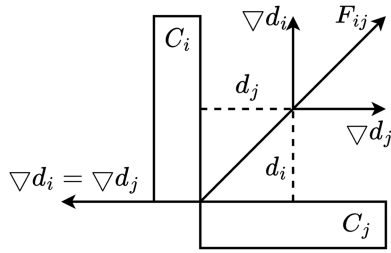


Fig. 1. Principle of GVG. The solid line represents S_{ij} the set of points equidistant to obstacles C_i and C_j . The dotted lines emphasize that at $x \in S_{ij}$, $d_i(x) = d_j(x)$.

- 1) The RRT* algorithm depends on the number of samples. Finding a high-quality path is difficult when the number of samples is small.
- 2) The sampling of the ordinary RRT* algorithm is blind, and its sampling in some positions of the map will not improve the path, but result in a slow convergence speed.
- 3) The fixed step RRT* algorithm has poor adaptability to special environments. The longer step size is poor in some narrow spaces, and the shorter step size takes a long time in open spaces.
- 4) The path solution obtained by the RRT* is composed of many line segments, which does not satisfy the constraints of the robot, and the robot cannot walk directly on the path.

The method proposed in this article aims to solve and improve the above-mentioned limitations. By adopting nonuniform sampling, the blindness of sampling is improved, and the sampling efficiency is improved. [Improved the limitations (1) (2)]. The environment information is recorded by storing the sampling circle, and the algorithm is adapted to various environments by adopting an adaptive step size. [Improved the limitation (3)]. By solving the OCP, we obtain a smooth trajectory while planning the velocity. [Solved limitation (4)].

III. PROPOSED ALGORITHM

In this section, we introduce the proposed TAA-RRT* algorithm with reference to the limitations of the RRT* algorithm proposed in the previous section.

A. Construction of the Target Area

The establishment of the TA is a key step in the entire algorithm. After the TA is established, the sampling area is limited to the TA. On the other hand, in the process of establishing the TA, the environment information is retained, and the narrow situation of the environment is recorded to facilitate the subsequent implementation of the adaptive step size strategy.

This section uses the GVG method to establish the TA. The workspace X is populated by convex obstacles $C_1, \dots, C_n$. It is assumed that the boundary of X is a collection of convex sets, which are members of the obstacle set C_i .

As shown in Fig. 1, the distance between a point x and a convex set C_i is termed the single object distance function and

Algorithm 1: Construction of TA.

Input: $x_{\text{start}}, x_{\text{goal}}, \text{Map}$
Output: GVGMap, TA

- 1: *ObstacleEdgeDetect*(*Map*)
- 2: (*GVGMap*, *DistanceToObs*) = *GVGInitial*(*Map*)
- 3: *ConnectToPoint*($x_{\text{start}}, x_{\text{goal}}, \text{GVGMap}$)
- 4: *InitialPath* = $A^*(\text{GVGMap})$
- 5: $\text{TA} = []$
- 6: **for** $i = 1 : \text{length}(\text{InitialPath})$
- 7: *CenterPoint* = *InitialPath*(i)
- 8: *Radius* = *DistanceToObs*(*InitialPath*(i))
- 9: *Circle* = *Circle*(*CenterPoint*, *Radius*)
- 10: $\text{TA} = [\text{TA}; \text{Circle}]$
- 11: **end**
- 12: **return** *GVGMap*, *TA*

is defined as

$$d_i(x) = \min_{c_0 \in C_i} \|x - c_0\| \quad (3)$$

where $\|\cdot\|$ is the two norm in R^m . The gradient of $d_i(x)$ is

$$\nabla d_i(x) = \frac{x - c_0}{\|x - c_0\|} \quad (4)$$

where c_0 is the nearest point to x in C_i .

Take the two-equidistant face as an example. If X is solely populated with disjoint convex obstacles, the two-equidistant surjective surface will be defined as

$$F_{ij} = \{x \in X : d_i(x) = d_j(x) \text{ and } \nabla d_i(x) \neq \nabla d_j(x)\}. \quad (5)$$

In the 2-D case, GVG contains a series of edges and vertices. If F_{ij} is a point, it is called a generalized voronoi vertex. A major feature of GVG is the function of its visual distance, and its specific calculation results can be obtained through measurement data.

With the basic knowledge of GVG above, we can proceed to the establishment of the TA, as shown in Algorithm 1. First, we should input $x_{\text{start}}, x_{\text{goal}}$ and map, and use the *ObstacleEdgeDetect* to detect the edge information of obstacles. Then draw a GVD and measure the distance from the GVG node to the nearest obstacle. The distance information is stored in *DistanceToObs*. We know that any two points on the map can be connected by connecting to GVG, so we use the function *ConnectToPoint* to connect x_{start} and x_{goal} to the GVG node. Then use A^* to find a shortest path along the GVG map. Take the point on this path as the center, delineate a circular area with the closest distance to the obstacle as the radius. Finally, the TA is obtained, and the specific process is shown in 6–11 of Algorithm 1. Importantly, the size of these circle radii can reflect the narrow information of the environment. The specific process is shown in Fig. 2(a)–(c).

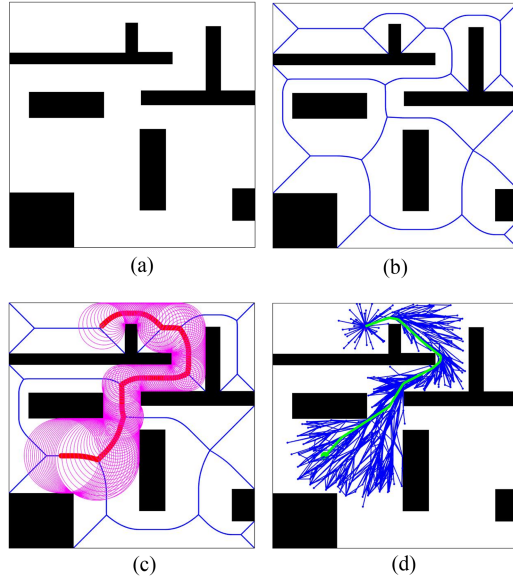


Fig. 2. TAA-RRT* Algorithmic Framework. (a) Enter an 800×800 map. (b) Blue line is the generalized voronoi diagram. (c) Red thick line is the initial path, and the pink circular area represents the target area. (d) Green line represents the last path found. (a) Map (b) GVGMMap. (c) Construct Target Area. (d) Nonuniform Sampling.

B. Nonuniform Sampling With Adaptive Step Size

To improve the efficiency, the TAA-RRT* only samples in the TA established in the previous section, making the algorithm's sampling more "focused," as shown in Fig. 2(d).

In addition, in the traditional RRT algorithm, the expansion step size is set in advance by the program, and then the tree is expanded with a fixed step size. However, the step size also has a great influence on the path finding time and path quality. When finding the initial path, a larger step size is easier to find the initial path in a short time, but in terms of path quality, a smaller step size is better for improving the path in each iteration process, especially when the environment is very narrow, this advantage will be more obvious. Therefore, combined with the environmental information retained when establishing the TA, the adaptive step size is set as follows:

$$S_{\text{step}} = \begin{cases} \text{Step}_{\text{max}}, & d > d_{\text{max}} \\ \frac{d-0}{d_{\text{max}}} \times \text{Step}_{\text{max}}, & 0 < d < d_{\text{max}} \end{cases} \quad (6)$$

where d represents the radius of the circle where the sampling point is located, d_{max} represents the environmental threshold, and Step_{max} represents the maximum expansion step.

C. Direct Connection Strategy

The direct connection strategy is adopted here. There are two main reasons for adopting this strategy: first, if the path is optimal, the DCS can clarify the exact mathematical representation of the path. Second, it can effectively improve the randomness of sampling algorithms and further optimize the path.

The path planning algorithm of the whole first layer is shown as in the Algorithm 2, the input of the algorithm not only has

x_{start} , x_{goal} , and the map, but also the maximum number of samples Itera and threshold d_{max} . First, the initialization of the tree and the establishment of the TA in Section A are performed. After entering the loop, each iteration must use the function *TASampling* to sample in the TA. According to the sampling point, the radius of the circle where it is located determines the corresponding sampling step, and the determined sampling step S_{step} is extended to generate new nodes x_{new} . The new node reselects the parent node among the nearby nodes, performs reconnection, and pruning operations, as shown in lines 13–20 and 23–30 of the algorithm, this part is also the process of TAA-RRT* optimization. Finally, the *DirectConnectionStrategy* is used to further optimize the path. In particular, the function *ObstacleFree* in the Algorithm 2 uses image processing methods for obstacle detection. For the algorithm, we need global map information. In practical applications, we first need to use radar sensors for mapping, and then for path planning.

D. Theoretical Analysis of Algorithms

1) Theoretical Properties: In this section, the main work is to carry out the following theoretical analysis.

Theorem 1: For a solution problem, when there are possible solutions, the probability of its solution is one

$$P(\sigma : [0, 1] \rightarrow X_{\text{free}}) = 1$$

$$\sigma(0) = x_{\text{start}}, \sigma(1) = x_{\text{goal}}. \quad (7)$$

Proof: The improved algorithm proposed in this article is based on the nonuniform sampling technique, which concentrates the sampled area in the established target area. The target area is established using a generalized voronoi diagram, which ensures the connectivity of the established map. Any two points on the map can be connected to nearby GVG nodes and then connected through the GVG map. It can be known from the previous chapters that there is at least one feasible path in the target area. Then by using an algorithm for forward incremental search, at least one feasible path can be found, that is to say, the algorithm is convergent, which proves the completeness of the algorithm. ■

Theorem 2: Let c^* be the path cost of the optimal path, and c_n be the path cost returned by the algorithm after sampling n times in the target area. When the algorithm reselects the parent node and reconnects, it uses the following radius to search for nearby nodes:

$$r_n = (1 + \eta) 2 \left(\frac{1}{d} \right)^{\frac{1}{d}} \left(\frac{\mu(X_{\text{free}})}{\zeta_d} \right)^{\frac{1}{d}} \left(\frac{\log(n)}{n} \right)^{\frac{1}{d}} \quad (8)$$

where η is a positive constant, d is the dimension of the space, X_{free} denotes the Lebesgue measure of the obstacle-free space, and ζ_d is the volume of the unit ball in the d -dimensional space.

When the number of samples is infinite, the path cost c_n will approach the optimal path cost c^* with probability 1

$$\lim_{n \rightarrow \infty} P(c_n = c^*) = 1. \quad (9)$$

Algorithm 2: TAA-RRT*.

Input: $x_{\text{start}}, x_{\text{goal}}, \text{Map}, \text{Itera}$
Output: $\text{GVGMap}, \text{TA}, \text{path } \sigma$

```

1:  $V \leftarrow x_{\text{start}}, E \leftarrow \emptyset, \sigma = (V, E)$ 
2:  $i = 1, \text{TA} = \text{ConstructionOfTA}(x_{\text{start}}, x_{\text{goal}}, \text{Map})$ 
3: while  $i < \text{Itera}$ 
4:    $x_{\text{rand}} = \text{TASampling}();$ 
5:    $x_{\text{nearest}} = \text{Nearest}(\sigma, x_{\text{rand}});$ 
6:    $S_{\text{step}} = d(x_{\text{rand}})$ 
7:    $x_{\text{new}} = \text{Steer}(x_{\text{nearest}}, x_{\text{rand}}, S_{\text{step}});$ 
8:    $\text{Cost}(x_{\text{new}}) = S_{\text{step}} + \text{Cost}(x_{\text{nearest}})$ 
9:   if  $!\text{ObstacleFree}(x_{\text{new}}, x_{\text{nearest}})$  then
10:     Continue;
11:   end if
12:    $X_{\text{near}} = \text{NearestNeighbors}(V, x_{\text{new}});$ 
13:   for  $x_{\text{near}} \in X_{\text{near}}$  do
14:     if  $\text{ObstacleFree}(x_{\text{near}}, x_{\text{new}})$  then
15:       if  $\text{Length}(x_{\text{near}}, x_{\text{new}}) + \text{Cost}(x_{\text{near}}) <$   

          $\text{Cost}(x_{\text{new}})$  then
16:          $\text{Cost}(x_{\text{new}}) =$   

          $\text{Length}(x_{\text{near}}, x_{\text{new}}) + \text{Cost}(x_{\text{near}})$ 
17:          $x_{\text{parent}} \leftarrow x_{\text{near}}$ 
18:       end if
19:     end if
20:   end for
21:    $V \leftarrow V \cup \{x_{\text{new}}\}$ 
22:    $E \leftarrow E \cup \{(x_{\text{new}}, x_{\text{parent}})\}$ 
23:   for  $x_{\text{near}} \in X_{\text{near}}$  do
24:     if  $\text{Length}(x_{\text{new}}, x_{\text{near}}) + \text{Cost}(x_{\text{new}}) < \text{Cost}(x_{\text{near}})$  then
25:        $\text{Cost}(x_{\text{near}}) = \text{Length}(x_{\text{new}}, x_{\text{near}}) + \text{Cost}(x_{\text{new}})$ 
26:        $x_{\text{parenttemp}} \leftarrow x_{\text{new}}$ 
27:       if  $x_{\text{parent}} == x_{\text{parenttemp}}$  then
28:          $E \leftarrow E - (x_{\text{near}}, \text{parent of } x_{\text{near}})$ 
29:          $E \leftarrow E \cup (x_{\text{near}}, x_{\text{new}})$ 
30:       end if
31:     end if
32:   end for
33: end while
34:  $\sigma_{\text{direct}} = \text{DirectConnectionStrategy}(\sigma)$ 
35: return  $\sigma, \sigma_{\text{direct}}$ 

```

Proof: First, when building the TA, use the A* to search for the shortest path on the GVG map, then build TA based on this path, because this path is the homotopy solution of the optimal path, our TA based on the minimum distance to the nearest obstacle contains the globally optimal path.

Since the RRT* algorithm itself has been proved to have asymptotic optimality, the improvement of this algorithm is to change the expansion step size. For asymptotic optimality, the step size have no effect on the RRT*, so the TAA-RRT* is asymptotically optimal. ■

Remark 1: Since TAA-RRT* is a global planning method, the algorithm itself does not have the characteristics of dealing with

dynamic obstacles, but when facing unknown obstacles, it can be processed by replanning or combined with other methods.

Theorem 3: For a path planning problem, when the number of samples is n , the time complexity of the proposed algorithm is $O(n \log(n))$, and the space complexity is $O(n \log(n))$.

Proof: When building the TA, the time complexity of obstacle edge detection and drawing GVG map and building TA are all less than or equal to $O(n)$. When using the A* for initial path finding, the computational complexity is $O(n \log n)$, so the computational complexity of the initial stage is $O(n \log n)$.

Then based on the RRT* framework, as shown in the algorithm, in each iteration, the new node will reselect the parent node and reconnect, so the computational complexity is $O(n \log n)$. Therefore, the complexity of the proposed algorithm is also $O(n \log n)$.

For the space complexity, it takes $O(n)$ space to store the target area TA, and each new node needs $O(n \log n)$ space when storing nearby nodes, so according to the big O rule, the proposed algorithm has a space complexity of $O(n \log n)$.

Therefore, the TAA-RRT* algorithm has the same complexity relative to the RRT* algorithm, but it can achieve a fast convergence speed and a good path solution. ■

2) Theoretical Comparative Analysis: In this section, we analyze the advantages of TAA-RRT* over the RRT* algorithm from the theoretical level. We will present the following two aspects:

Theorem 4: For the case where the number of sampling points is n , the number n_{taa} of TAA-RRT* sampling distribution in the homotopy region of the optimal solution is more than the number n_{rrt} of RRT*.

Proof: The sampling algorithm continuously optimizes the path through each sampling. When the sampling points are distributed in the area where the optimal path is located, we call it an “effective point” that has an effect on improving the path. We found that under the same sampling number n of the two algorithms, the effective points n_{rrt} of RRT* and the effective points n_{taa} of TAA-RRT* have the following relationship:

$$\begin{cases} n_{\text{rrt}} = \frac{S_{\text{TA}}}{S_{\text{map}}} \times n \\ n_{\text{taa}} = n \end{cases} \Rightarrow n_{\text{taa}} \geq n_{\text{rrt}} \quad (10)$$

where S_{TA} is the area of the homologation area and S_{map} is the area of the whole map. The relationship between the two is $S_{\text{TA}} \leq S_{\text{map}}$. Therefore, this also proves its high efficiency. ■

Theorem 5: TAA-RRT* utilizes DCS strategy for secondary optimization, which converges faster than RRT*.

Proof: TAA-RRT* introduces the DCS strategy after optimization. We assume that there are j original path segments in the i th path optimized by DCS, so the relationship between the front and rear path lengths is as follows:

$$\text{Length}(\sigma_{\text{direct}}(i)) \leq \sum_{k=1}^j \text{Length}(\sigma(k)). \quad (11)$$

Combined with the algorithm complexity analysis, in the case of the same algorithm complexity as RRT*, TAA-RRT* can achieve a fast and high-quality solution with fewer sampling points. Therefore, its fast convergence and stability are proved. ■

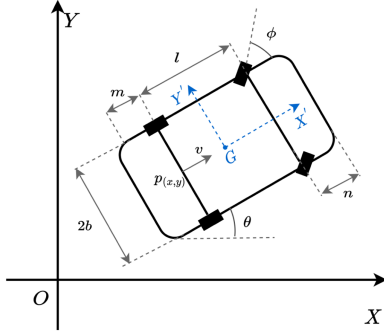


Fig. 3. Vehicle model diagram.

IV. TRAJECTORY OPTIMIZATION MODEL FRAMEWORK

A. Motivations

The last chapter described the use of the TAA-RRT* algorithm to find the optimal path. Then, use the DCS and smooth a part of the polyline segment, but there are still straight corners, and the AGV cannot drive at the corner due to its own constraints. In order to realize that the AGV travels on the path, the straight line segment is used as the path constraint, and then the speed on the straight line segment is planned, and the corner area is established to optimize the trajectory at the corner. Thus, the second-layer trajectory planning problem is transformed into an OCP for solving.

B. Vehicle Motion Model

To describe the motion of the AGV, the structure of the AGV is shown in Fig. 3 and the equation of vehicle motion is as follows:

$$\begin{cases} \dot{p}_x(t) = v(t) \cos(\theta(t)) \\ \dot{p}_y(t) = v(t) \sin(\theta(t)) \\ \dot{v}(t) = a(t) \\ \dot{a}(t) = \eta(t) \\ \dot{\theta}(t) = v(t) \tan(\phi(t))/l \\ \dot{\phi}(t) = \omega(t) \end{cases} \quad (12)$$

where (p_x, p_y) represents the center point of the rear wheel, t represents time, v and a represent the speed and acceleration of the car, respectively, θ represents the direction angle, and ϕ represents the steering angle of the steering wheel. $jerk$ and ω represent the jerk and the angular velocity of the steering wheel, respectively.

So the state x and the control u are respectively defined as $x = [p_x, p_y, v, a, \theta, \phi]^T$, $u = [\eta, \omega]$. Other vehicle parameters are the front overhang length n , the length l between the front and rear wheels, the rear overhang length m and the vehicle width $2b$. In this work, we temporarily regard the vehicle as a mass point in the map, without considering the shape and size of the vehicle. To simplify the description, we can abbreviate (11) as $\dot{x}(t) = f(x(t), u(t))$.

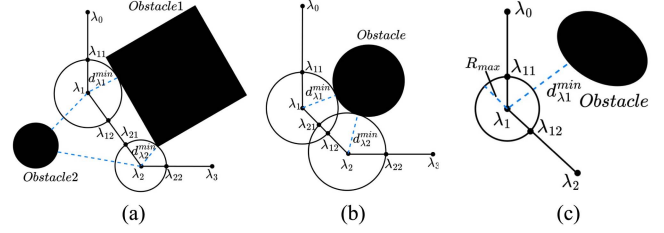


Fig. 4. Three cases of path constraints. (a) Turning areas of the vehicles do not intersect. (b) Intersection between turn areas. (c) Waypoint is far from the obstacle.

C. Vehicle Motion Constraints

1) State and Control Constraints: The range of state and control constraints for each state is as follows:

$$\begin{aligned} p_x^{\min} &\leq p_x(t) \leq p_x^{\max}, p_y^{\min} \leq p_y(t) \leq p_y^{\max} \\ v^{\min} &\leq v(t) \leq v^{\max}, a^{\min} \leq a(t) \leq a^{\max} \\ \theta^{\min} &\leq \theta(t) \leq \theta^{\max}, \phi^{\min} \leq \phi(t) \leq \phi^{\max} \\ d_a^{\min} &\leq \eta(t) \leq d_a^{\max}, d_k^{\min} \leq \dot{k}(t) \leq d_k^{\max} \end{aligned} \quad (13)$$

where k represents the curvature value at time t , $k = \tan(\theta)/l$. $\dot{k} = \omega/(l \cos^2(\theta))$ is the derivative of k .

2) Path Constraint: This section clarifies the path constraints of the AGV on the straight segment and the corner area constraints. The corner area is established as a circular area without obstacles. It mainly involves the following three situations:

Case 1: The first case is shown in Fig. 4(a), where λ_i represents the waypoint processed by the first layer. The centers of the established turn areas are these waypoints. The radius $d_{\lambda_i}^{\min}$ is the minimum distance from waypoint λ_i to the obstacle. λ_{ij} is the intersection of the circular area and the straight line segment. When the two circles do not intersect, the path constraint is three straight line segment constraints and two corner constraints.

Case 2: The second case is shown in Fig. 4(b). If the circles in the turning area intersect, the path constraints in the situation in the figure are two straight line segment constraints and a turning area. At this time, the turning area is the union of the two circular areas.

Case 3: The third case is shown in Fig. 4(c). The minimum distance $d_{\lambda_1}^{\min}$ is larger, which is greater than the threshold value $R_{\max}$. In this case, the radius of the turning area is set to $R_{\max}$ in order to retain a long straight line segment.

Therefore, considering these three cases, the path constraints processed in the first layer can be obtained by the Algorithm 3. The function needs to input the path σ_{direct} , and finally obtains the path constraint set Ψ . First, you need to calculate the number of nodes on the path σ_{direct} , and then use the function *CalculateJunction* to calculate the size of the turning area at the turning point and the intersection with the straight line segment. By judging and calculating one by one, then the constraint expression is calculated as shown in 4–13 of Algorithm 3, the

Algorithm 3: Path Constraint Solver.

Input: σ_{direct}
Output: *Constraint set* Ψ

```

1:  $j = 1; k = 1;$ 
2:  $[\lambda_0, \dots, \lambda_n] = \text{ReadPoints}(\sigma_{\text{direct}});$ 
3:  $[W, D] = \text{CalculateJunction}([\lambda_1, \dots, \lambda_{n-1}]);$ 
4:  $L_0 = \lambda_0 \lambda_{11} \cup \lambda_{n-1,2} \lambda_n;$ 
5: for  $i = 1 : n - 1$ 
6:   if DistanceJudge then
7:      $L_j = \lambda_{i,2} \lambda_{i+1,1}$ 
8:      $C_k = \text{Circle}(\lambda_i)$ 
9:      $j = j + 1; k = k + 1;$ 
10:  else
11:     $C_k = C_k \cup \text{Circle}(\lambda_i) \cup \text{Circle}(\lambda_{i+1})$ 
12:  end if
13: end for
14:  $\Psi = [L_0, L_1, \dots, L_j, C_1, \dots, C_k];$ 
15: return  $\Psi$ 

```

straight line segment constraints are placed in the L set, and the turning area constraints are placed in the C set.

Taking two points λ_i and λ_j as an example, suppose $d_{\lambda_i}^{\min}$ is less than or equal to $R_{\max}$. The algorithm mainly involves the following three formulas:

$$\sqrt{(x_{\lambda_i} - x_{\lambda_j})^2 + (y_{\lambda_i} - y_{\lambda_j})^2} > d_{\lambda_i}^{\min} + d_{\lambda_j}^{\min} \quad (14)$$

$$y - y_{\lambda_i} = \frac{y_{\lambda_j} - y_{\lambda_i}}{x_{\lambda_j} - x_{\lambda_i}}(x - x_{\lambda_i}) \quad (15)$$

$$\sqrt{(x_{\lambda_i} - x_{\lambda_j})^2 + (y_{\lambda_i} - y_{\lambda_j})^2} > d_{\lambda_i}^{\min}. \quad (16)$$

Among them, *DistanceJudge* uses formula (12) to judge whether two adjacent turning areas intersect. In the algorithm, the shape such as $\lambda_i \lambda_j$ represents the straight line constraint, and the specific mathematical form is shown in formula (13). *Circle* represents the turning area constraint, and the specific form is shown in formula (14). All path constraints are placed in Ψ .

3) Boundary Constraints: The vehicle starts from a zero initial state, and when it reaches the end, the vehicle stops again. That is to say, the boundary constraints should satisfy the following conditions:

$$\begin{cases} v(0) = 0; \\ a(0) = 0; \\ v(t_f) = 0; \\ a(t_f) = 0. \end{cases} \quad (17)$$

D. Minimization Objective and Overall Optimization Formula

The second stage planning expects to achieve the planning with the minimum time on the shortest path obtained in the first stage, so t_f is chosen as the minimization objective function. The objective function forms the trajectory optimization model together with kinematic constraints, path constraints, and

boundary constraints

$$\begin{aligned} \min J &= t_f \\ \text{s.t. } \forall t \in [0, t_f] \\ \dot{x}(t) &= f(x(t), u(t)) \\ x(0) &= x_0, x(t_f) = x_f \\ x(t) &\in [x^{\min}, x^{\max}], u(t) \in [u^{\min}, u^{\max}]. \\ \text{path constraint } \Psi \end{aligned} \quad (18)$$

Among them, x_0 and x_f represent the initial state and the final state, respectively. Similarly, $x^{\min}, x^{\max}, u^{\min}, u^{\max}$ are boundary values representing state and control quantities, which is a simplified form of (12). Here we do not consider obstacle collision constraints, first because we treat the car as a particle in this work. Second, we have considered obstacle avoidance in the path planning of the first layer, and the turning area established by the second layer is also an obstacle-free area.

For the optimal control problem in (18), we use the Gaussian pseudospectral method of numerical methods to solve it. In this article, the Gaussian pseudospectral method is used to discretize the OCP using Gaussian points in $[-1, 1]$ as collocation points. After discretizing the problem, the nonlinear programming (NLP) problems is solved using the well-known sequential quadratic programming (SQP) algorithm, which is a powerful tool for solving NLP problems, especially for small-scale dense problems, with high convergence speed and accuracy. After solving using Gaussian pseudospectral method, we can obtain the optimal trajectory solution of the OCP.

E. Tracking the Planned Trajectory

To achieve the task, we need to use a controller to track the optimal trajectory. The tracking method used in this article is a linear quadratic regulator (LQR). We consider the following discrete kinematics model:

$$X_e(k+1) = AX_e(k) + Bu(k) \quad (19)$$

where $X_e = [p_x - x_r, p_y - y_r, \theta - \theta_r]^T$ represents the error of the state quantity, and $u = [v - v_r, \phi - \phi_r]^T$ represents the error of the control quantity. The matrices A and B are as follows:

$$A = \begin{bmatrix} 1 & 0 & -Tv_r \sin \theta_r \\ 0 & 1 & Tv_r \cos \theta_r \\ 0 & 0 & 1 \end{bmatrix}, B = \begin{bmatrix} T \cos \theta_r & 0 \\ T \sin \theta_r & 0 \\ \frac{T \tan \phi_r}{l} & \frac{T v_r}{l \cos^2 \phi_r} \end{bmatrix} \quad (20)$$

where T represents the sampling time. $[x_r, y_r, \theta_r, v_r, \phi_r]$ represents reference trajectory information.

In order to ensure that the trajectory can be tracked accurately, we need the following performance indicators to be optimal:

$$J = \sum_{k=1}^N (X_e^T Q X_e + u^T R u) \quad (21)$$

where Q denotes a semipositive definite state weighting matrix and R denotes a semipositive definite control weighting matrix.

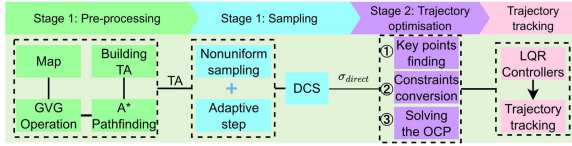


Fig. 5. Method framework diagram.

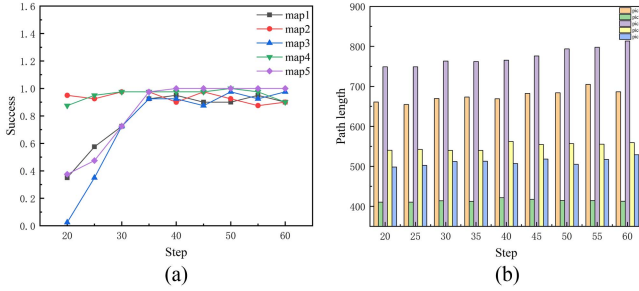


Fig. 6. Success rate of finding the initial path with a smaller number of samples at different step sizes and the corresponding path cost.

Therefore, according to the LQR control, we can set the control law as follows:

$$u = -(R + B^T P B)^{-1} B^T P A X_e \quad (22)$$

where P is the solution to the following Riccati equation:

$$P = A^T P A - A^T P B (R + B^T P B)^{-1} B^T P A + Q. \quad (23)$$

Therefore, the two-layer method framework is shown in Fig. 5.

V. EXPERIMENTS AND ANALYSIS

In this section, we conduct extensive experiments to demonstrate the superiority of the proposed method. The planning algorithms in our experiments are all simulated in MATLAB R2016b on Windows 10, and our computer is configured as Intel-i7 with 8 G RAM memory. Physical experiments are carried out in a real environment to verify the effectiveness of the proposed method.

A. Parameter Analysis

In the TAA-RRT* algorithm, the adaptive step size threshold $d_{\max}$ is a key factor affecting the rapidity of finding an initial path. We used Algorithm 4 to determine the appropriate step size threshold $d_{\max}$. First, we test the success rate and path cost of finding the initial path with different sampling steps in the case of a small number of samples in five test maps. We tested each set of data 50 times separately to calculate the success rate and used the average value for comparison of the path cost.

By setting $d_{\max}$ to different step lengths, we statistically obtained the comparison results shown in Fig. 6. Fig. 6(a) shows the success rate of finding the initial path, we can see that the longer the step length the easier it is to find the initial path, but as shown in Fig. 6(b) the corresponding initial path cost also increases. To ensure a 90% success rate in finding the initial path, we filter out steps below 90%. At the same time, it is considered

Algorithm 4: Selection of $d_{\max}$.

Input: Map, the step size set D and the success rate threshold.

Output: $d_{\max}$, the success rate and initial path cost of each step.

- 1: Step 1: Use fewer sampling points to calculate the success rate and path cost of the initial path at each step.
- 2: Step 2: Filter out steps with a low success rate.
- 3: Step 3: Compare the two performance metrics of step size and initial path cost. Choose the optimal step size as $d_{\max}$.

to make the initial path cost and step size as small as possible, so that the path can be optimized with small step size details. Therefore, in this experiment, we set $d_{\max} = 35$.

B. Comparative Simulations Analysis

In the simulation process, in order to ensure a fair comparison, each algorithm uses the same subroutines such as nearest neighbor search, collision detection, and data processing. Here we compare the TAA-RRT* algorithm with the advanced RRT* algorithm, the Informed-RRT* algorithm, and GVD-RRT* algorithm, where GVD-RRT* represents a class of nonuniform sampling methods based on the GVD method. The expansion step size for the RRT*, Informed-RRT*, and GVD-RRT* algorithms is set to 35. In each case, all planning algorithms were performed 50 times, and the final data were expressed in mean form.

The five maps are all 500×500 in size and represent traps, narrow passages, mazes, open environments, and cluttered environments. Fig. 7(a)–(d) (represented by map1) shows that the target point or origin exists in a trap area. Fig. 7(e)–(h) (represented by map2) shows that the path solution exists in the narrow channel. Fig. 7(i)–(l) (represented by map3) shows that the path solution exists in a maze-like scene. Fig. 7(m)–(p) (represented by map4) shows that the start and target points exist in the open environment. Fig. 7(q)–(t) (represented by map5) shows that the start and target points exist in clutter space.

Fig. 7 shows the results of each algorithm under 2000 samples. The path length, the number of samples, and the node utilization are used for evaluation, as shown in Fig. 7 and Table I. In Fig. 7, the sampling exploration of the TAA-RRT* algorithm is concentrated in the TA. It is confirmed that the TAA-RRT* algorithm is efficient, and it only concentrates on the effective area for sampling.

Fig. 8 shows the comparison results of the path lengths obtained after running the four algorithms for 50 times in the above five scenarios. Obviously, the path quality generated by TAA-RRT* algorithm is better than the other three algorithms. This advantage is more pronounced in narrow passages. At the same time, compared with the TAA-RRT* algorithm, the RRT*, Informed-RRT*, and GVD-RRT* algorithms have volatility, which also proves that the TAA-RRT* algorithm has better stability.

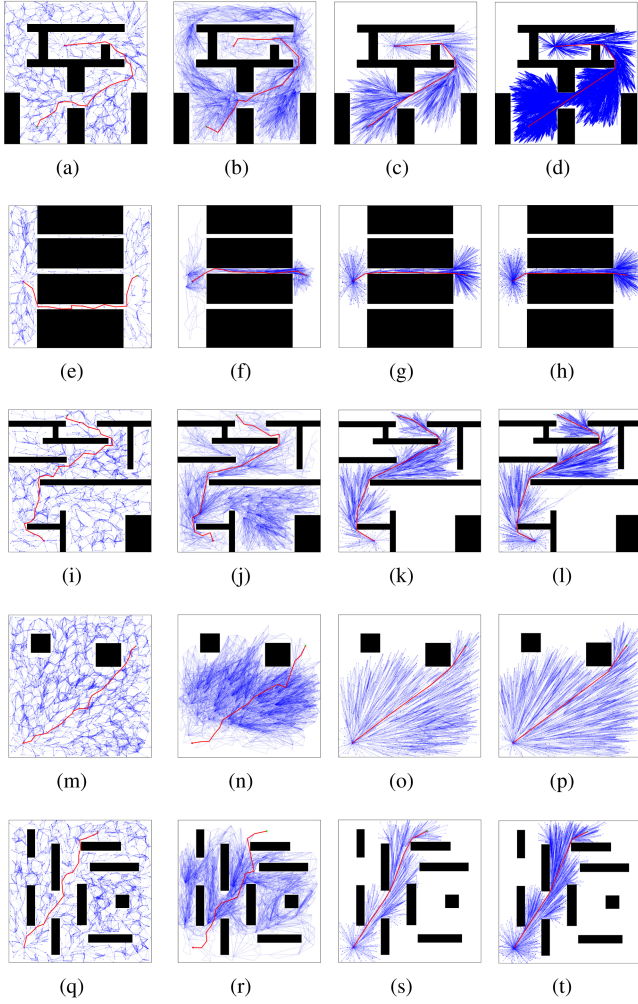


Fig. 7. Compare the performance of other algorithms in five scenarios. From left to right in the columns, the sampling algorithms are RRT*, Informed-RRT*, GVD-RRT*, and TAA-RRT*. The red route is the path found by the algorithm.

TABLE I
ALGORITHM CONVERGENCE RESULTS

map	algorithm	samples	path length	node utilization
map1	TAA-RRT*	1000	641.67	86.4%
map1	GVD-RRT*	1500	644.32	72.7%
map1	Informed-RRT*	8000	646.65	56.2%
map1	RRT*	14000	646.54	54%
map2	TAA-RRT*	1000	408.72	84.3%
map2	GVD-RRT*	1500	409.56	68.1%
map2	Informed-RRT*	5000	412.23	49.4%
map2	RRT*	15000	413.17	42.6%
map3	TAA-RRT*	1000	725.58	84.8%
map3	GVD-RRT*	1500	726.66	76.1%
map3	Informed-RRT*	8000	726.33	62.5%
map3	RRT*	20000	725.46	66.2%
map4	TAA-RRT*	1300	526.87	89.4%
map4	GVD-RRT*	2000	529.99	82.6%
map4	Informed-RRT*	8000	530.46	80.3%
map4	RRT*	20000	530.79	84.3%
map5	TAA-RRT*	1000	493.23	83.6%
map5	GVD-RRT*	1600	495.04	70.3%
map5	Informed-RRT*	5000	495.42	60.3%
map5	RRT*	20000	496.31	56.4%

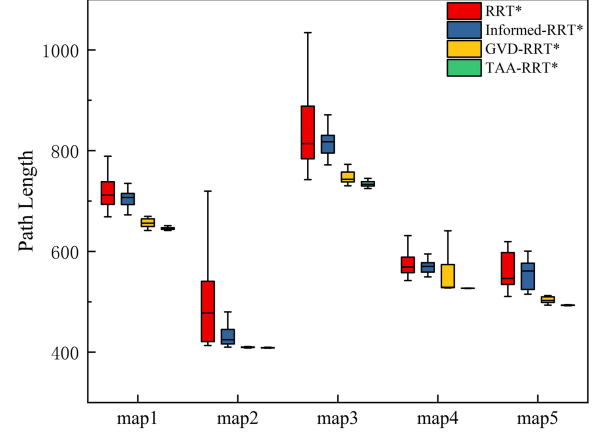


Fig. 8. Comparison of path lengths in five scenarios.

Table I records the number of samples, path length, and node utilization when each map converges to the optimal solution. Node utilization represents the percentage of the number of sampling nodes that can be expanded to the total number of samples. The number of samples can also reflect the convergence time information to a certain extent. It can be seen that in any case, TAA-RRT* can converge to the optimal solution with less number of sampling points, shorter time, and higher node utilization. Taking map1 as an example, when the TAA-RRT* algorithm reaches a stable convergence effect, the number of sampling points is reduced by 33.3% compared with GVD-RRT*, reduced by 87.5% compared with informed-RRT*, and reduced by 92.9% compared with RRT*. In the process of converging to the optimal solution, the node utilization efficiency of TAA-RRT* has been greatly improved compared with RRT* and Informed-RRT*. Moreover, for the GVD-RRT* algorithm with the same nonuniform sampling, the TAA-RRT* algorithm introduces a step size adaptive strategy, so that the node utilization efficiency has also been significantly improved. This also leads to the best path solution quality of the TAA-RRT* algorithm when converging to the optimal solution. It can be seen that the TAA-RRT* algorithm converges to the optimal solution at a faster speed and with higher efficiency.

Since TAA-RRT* introduces an adaptive step size strategy, here we compare the path length results between adaptive step size and fixed step size in the above four maps except the open environment. The fixed step size is set to 35. Fig. 9(a)–(d) shows the effect of different step sizes on the path length with a small number of samples. It can be seen from the figure that the introduction of the adaptive step size strategy has a certain effect on improving the path quality. Especially in some narrow spaces, the fixed step size cannot effectively improve the path, and the adaptive step size can slightly improve the path.

In conclusion, the simulation experiments demonstrate the good performance of the TAA-RRT* algorithm in different environments.

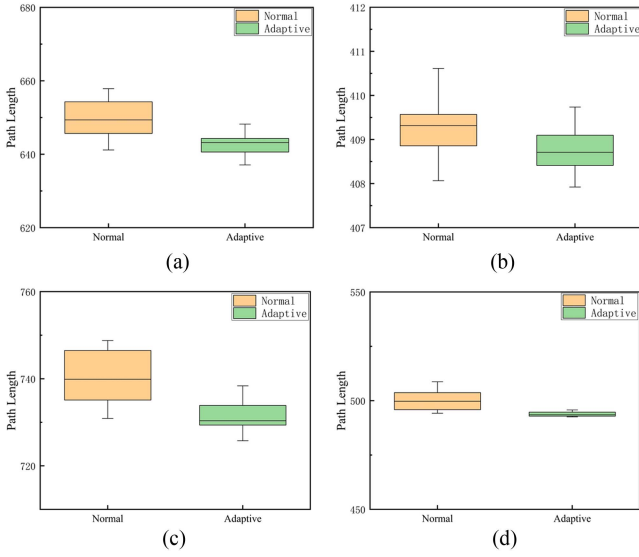


Fig. 9. Effects of adaptive step size strategies in four scenarios.

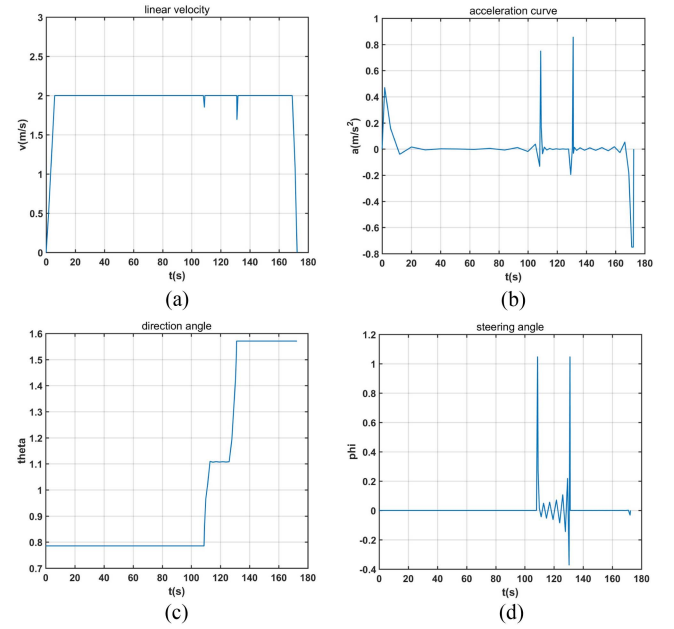


Fig. 11. State information of the trajectory planning result.

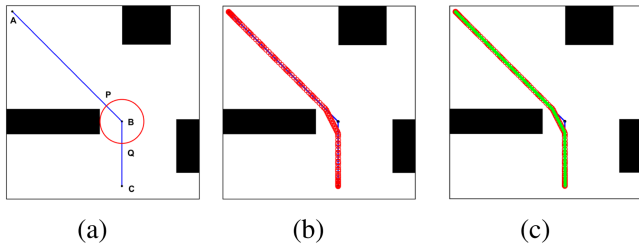


Fig. 10. Simulation scene and two-layer path results. (a) Blue broken line represents the path solution obtained by TAA-RRT*. (b) Red circles indicate the optimized trajectory points. (c) Green curve is the tracking result.

TABLE II
TIME INFORMATION FOR EACH STAGE

stage	1	2	3	J
time/s	(0,108.69)	(108.69,131.07)	(131.07,172.40)	172.40

C. Trajectory Optimization and Tracking Results

In order to verify the feasibility of the second-layer trajectory planning, we conduct a simulation experiment of trajectory optimization in a simulation scenario as shown in Fig. 10. The size of the map scene is 300×300 , and the AGV is regarded as a point in the map. The speed and acceleration of the unmanned vehicle are constrained in $v \in [-2, 2]$, $a \in [-0.9, 0.9]$. As shown in Fig. 10(a), the start and end points of the map are set to (10, 10), (180, 280), respectively. The blue broken line in Fig. 10(a) is the feasible path obtained by using the TAA-RRT* algorithm in this scenario with a small number of adoptions. The red circle is the boundary of the corner optimization area. Next, we establish OCP on this basis and solve it.

After solving the OCP, the optimal result is finally obtained, as shown in Table II. The optimal performance index $J = 172.4s$. Fig. 11(a)–(d) shows the speed, acceleration, direction angle,

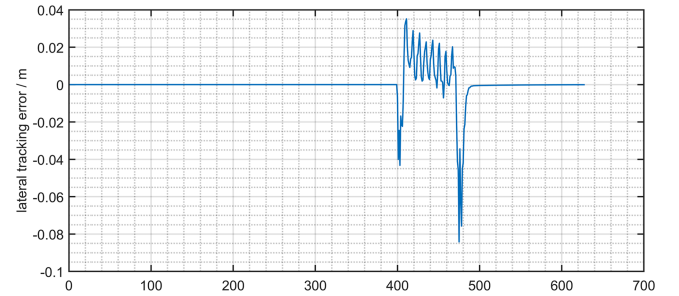


Fig. 12. Vehicle tracking lateral error simulation results.

and steering angle of the AGV as a function of time. It can be seen that the speed, acceleration, direction, and steering of the AGV have changed significantly at the turning points P and Q. This is because the AGV decelerates when changing direction. From the experimental results, the vehicle runs relatively smoothly, and the final trajectory is shown in Fig. 10(b). It can be seen that the trajectory points on the straight line segment are indeed distributed on the straight line path planned by the first layer. Therefore, it can be shown through experiments that the method proposed is effective. Finally, using the tracking method introduced earlier in this article, the tracking results are shown in Fig. 10(c). In the simulation experiment of tracking, we set the initial position at the starting point, and the lateral error of tracking is shown in Fig. 12. It can be seen that the trajectory can basically achieve accurate tracking.

D. Physics Experiments

In order to test the effectiveness of the method proposed in this article, we conducted a physical experiment. The physical

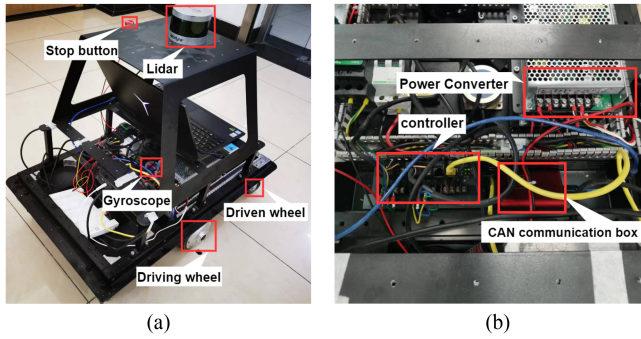


Fig. 13. Experiment platform.

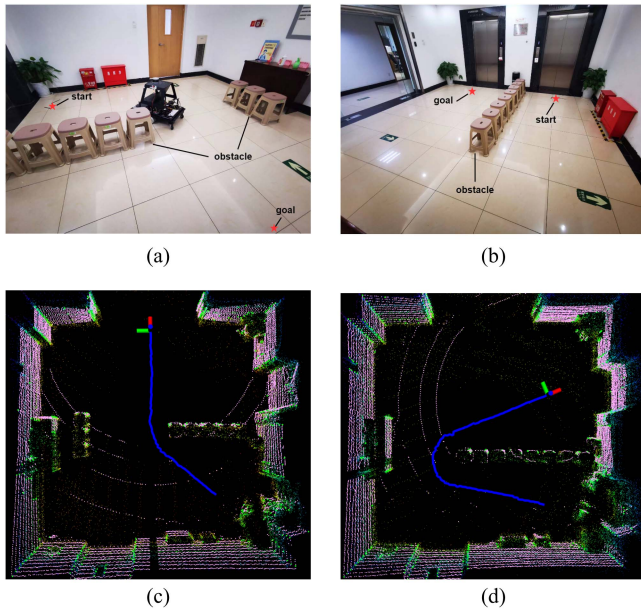


Fig. 14. Real experimental scene.

platform is shown in Fig. 13. Here, a laptop is used for command control, the unmanned vehicle is equipped with a lidar, and a TL740D high-precision gyro angle meter. As shown in Fig. 13(a), our unmanned vehicle has two driving wheels and four driven wheels. Similar to the method in [34], as a global planning method, we need to use the radar sensors on the unmanned vehicle to carry out environmental mapping operations in order to obtain global static environmental information. During the experiment, the maximum speed of the unmanned vehicle is 0.2 m/s, and the maximum acceleration is 0.5 m/s.

We set up two different scenarios here to verify the effectiveness of the proposed method. Fig. 14(a) shows the first scene, which is the experimental scene set by the trajectory optimization in the previous section. The stool and table in the picture are static obstacles arranged. Fig. 14(c) is a schematic diagram of the result of the running trajectory in Rviz. It can be seen that the trajectory of the car in the actual operation process is also roughly divided into the straight-line driving stage and the turning stage. Fig. 14(b) shows the second experimental scene, this time we set a longer obstacle block, so that the unmanned

vehicle passes through two longer straight sections and a turning process, and passes through a narrow passage. It can be seen that although the vehicle will be subject to some disturbances during driving, it can track our planned trajectory through tracking control. This further confirms the reliability of the method. Our demonstration video is put on.¹

VI. CONCLUSION

This article proposed a two-layer optimal trajectory planning method. The first layer used a sampling-based path planning algorithm to find the optimal path. In order to improve the efficiency of the sampling algorithm, the sampling target area was established based on the GVG graph, which effectively improved the convergence speed of the algorithm. An adaptive step size and a direct connection strategy were introduced, which effectively improved the quality of the path. According to various simulation experiments, the proposed TAA-RRT* algorithm had certain efficiency and stability. In order to solve the problem that the sampling-based algorithm cannot meet the constraints of the robot, this article introduced the second-layer trajectory optimization method, and solved a trajectory that the robot can run by solving the OCP.

In practical applications, there were often some unknown obstacles in the scene during AGV driving, which will directly affect its operating safety. In the future, a work worth of research is to improve the proposed method by combining the local path method or artificial potential field method, so that it can avoid unknown obstacles in the environment.

REFERENCES

- [1] J. Kim, "Trajectory generation of a two-wheeled mobile robot in an uncertain environment," *IEEE Trans. Ind. Electron.*, vol. 67, no. 7, pp. 5586–5594, Jul. 2020.
- [2] A. V. Le et al., "Towards optimal hydro-blasting in reconfigurable climbing system for corroded ship hull cleaning and maintenance," *Expert Syst. Appl.*, vol. 170, 2021, Art. no. 114519.
- [3] W. Chi, C. Wang, J. Wang, and M. Q.-H. Meng, "Risk-DTRRT-based optimal motion planning algorithm for mobile robots," *IEEE Trans. Autom. Sci. Eng.*, vol. 16, no. 3, pp. 1271–1288, Jul. 2019.
- [4] R. Chai et al., "Multiobjective overtaking maneuver planning for autonomous ground vehicles," *IEEE Trans. Cybern.*, vol. 51, no. 8, pp. 4035–4049, Aug. 2021.
- [5] B. Siciliano and O. Khatib, "Robotics and the handbook," in *Springer Handbook of Robotics*. New York, NY, USA: Springer, 2016, pp. 1–6.
- [6] X. Bai, W. Yan, and M. Cao, "Clustering-based algorithms for multivehicle task assignment in a time-invariant drift field," *IEEE Robot. Autom. Lett.*, vol. 2, no. 4, pp. 2166–2173, Oct. 2017.
- [7] J. Qi, H. Yang, and H. Sun, "MOD-RRT*: A sampling-based algorithm for robot path planning in dynamic environment," *IEEE Trans. Ind. Electron.*, vol. 68, no. 8, pp. 7244–7251, Aug. 2021.
- [8] J. J. M. Lunenburg, S. A. M. Coenen, G. J. Naus, M. J. G. van de Molengraft, and M. Steinbuch, "Motion planning for mobile robots: A method for the selection of a combination of motion-planning algorithms," *IEEE Robot. Autom. Mag.*, vol. 23, no. 4, pp. 107–117, Dec. 2016.
- [9] Z. Jian, S. Zhang, S. Chen, Z. Nan, and N. Zheng, "A global-local coupling two-stage path planning method for mobile robots," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 5349–5356, Jul. 2021.
- [10] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, Jul. 1968.
- [11] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," in *Proc. IEEE Int. Conf. Robot. Autom.*, 1985, vol. 2, pp. 500–505.

¹<https://youtu.be/ANNTPrnPR2A>

- [12] E. Masehian and D. Sedighzadeh, "A multi-objective pso-based algorithm for robot path planning," in *Proc. IEEE Int. Conf. Ind. Technol.*, 2010, pp. 465–470.
- [13] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE Trans. Robot. Autom.*, vol. 12, no. 4, pp. 566–580, Aug. 1996.
- [14] S. M. LaValle and J. J. Kuffner Jr., "Randomized kinodynamic planning," *Int. J. Robot. Res.*, vol. 20, no. 5, pp. 378–400, 2001.
- [15] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Int. J. Robot. Res.*, vol. 30, no. 7, pp. 846–894, 2011.
- [16] W. Chi, Z. Ding, J. Wang, G. Chen, and L. Sun, "A generalized voronoi diagram-based efficient heuristic path planning method for RRTs in mobile robots," *IEEE Trans. Ind. Electron.*, vol. 69, no. 5, pp. 4926–4937, May 2022.
- [17] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Informed RRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2014, pp. 2997–3004.
- [18] L. Zhang and D. Manocha, "An efficient retraction-based RRT planner," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2008, pp. 3743–3750.
- [19] J. Xu, K. Song, D. Zhang, H. Dong, Y. Yan, and Q. Meng, "Informed anytime fast marching tree for asymptotically optimal motion planning," *IEEE Trans. Ind. Electron.*, vol. 68, no. 6, pp. 5068–5077, Jun. 2021.
- [20] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Batch informed trees (BIT): Sampling-based optimal planning via the heuristically guided search of implicit random geometric graphs," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2015, pp. 3067–3074.
- [21] O. Salzman and D. Halperin, "Asymptotically near-optimal RRT for fast, high-quality motion planning," *IEEE Trans. Robot.*, vol. 32, no. 3, pp. 473–483, Jun. 2016.
- [22] J. Wang, M. A.-H. Meng, and O. Khatib, "EB-RRT: Optimal motion planning for mobile robots," *IEEE Trans. Autom. Sci. Eng.*, vol. 17, no. 4, pp. 2063–2073, Oct. 2020.
- [23] B. An, J. Kim, and F. C. Park, "An adaptive stepsize RRT planning algorithm for open-chain robots," *IEEE Robot. Autom. Lett.*, vol. 3, no. 1, pp. 312–319, Jan. 2018.
- [24] A. Yershova, L. Jaillet, T. Siméon, and S. M. LaValle, "Dynamic-domain RRTs: Efficient exploration by controlling the sampling domain," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2005, pp. 3856–3861.
- [25] P. Polack, F. Althé, B. d'Andréa Novel, and A. de La Fortelle, "The kinematic bicycle model: A consistent model for planning feasible trajectories for autonomous vehicles?," in *Proc. IEEE Intell. Veh. Symp.*, 2017, pp. 812–818.
- [26] M. Kobayashi and N. Motoi, "Local path planning: Dynamic window approach with virtual manipulators considering dynamic obstacles," *IEEE Access*, vol. 10, pp. 17018–17029, 2022.
- [27] Z. Yongzhe, B. Ma, and C. K. Wai, "A practical study of time-elastic-band planning method for driverless vehicle for auto-parking," in *Proc. IEEE Int. Conf. Intell. Auton. Syst.*, 2018, pp. 196–200.
- [28] C. You, J. Lu, D. Filev, and P. Tsiotras, "Autonomous planning and control for intelligent vehicles in traffic," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 6, pp. 2339–2349, Jun. 2020.
- [29] B. Li, Y. Ouyang, L. Li, and Y. Zhang, "Autonomous driving on curvy roads without reliance on frenet frame: A cartesian-based trajectory planning method," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 9, pp. 15729–15741, Sep. 2022.
- [30] B. Li et al., "Mixed-integer and conditional trajectory planning for an autonomous mining truck in loading/dumping scenarios: A global optimization approach," *IEEE Trans. Intell. Veh.*, to be published, doi: [10.1109/TIV.2022.3214777](https://doi.org/10.1109/TIV.2022.3214777).
- [31] B. Li et al., "Optimization-based trajectory planning for autonomous parking with irregularly placed obstacles: A lightweight iterative framework," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 8, pp. 11970–11981, Aug. 2022.
- [32] H. Choset and J. Burdick, "Sensor-based exploration: The hierarchical generalized voronoi graph," *Int. J. Robot. Res.*, vol. 19, no. 2, pp. 96–125, 2000.
- [33] D. Kim, J. Lee, and S.-E. Yoon, "Cloud RRT: Sampling cloud based RRT," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2014, pp. 2519–2526.
- [34] J. Wang and M. Q.-H. Meng, "Optimal path planning using generalized voronoi graph and multiple potential functions," *IEEE Trans. Ind. Electron.*, vol. 67, no. 12, pp. 10621–10630, Dec. 2020.



Runda Zhang received the B.S. degree in traffic equipment and control engineering from Taiyuan University of Technology, Taiyuan, China, in 2021. He is currently working toward the M.S. degree in control science and engineering with Beijing Institute of Technology, Beijing, China.

His research interests include path planning and trajectory optimization for robots.



Runqi Chai (Member, IEEE) received the B.S. degree in information and computing science from the North China University of Technology, Beijing, China, in 2015 and the Ph.D. degree in aerospace engineering from Cranfield University, Cranfield, U.K., in August 2018.

He is currently a Professor with Beijing Institute of Technology. His research interests include unmanned vehicle trajectory optimization, guidance, and control.



Senchun Chai (Senior Member, IEEE) received the Ph.D. degree in networked control system from the University of South Wales, Pontypridd, U.K., in 2007.

He is currently a Professor with the School of Automation, Beijing Institute of Technology, Beijing, China. His current research interests focus on flight control system, networked control systems, embedded systems, and multiagent control systems.



Yuanqing Xia (Senior Member, IEEE) received the B.S. degree in fundamental mathematics from the Department of Mathematics, Chuzhou University, Chuzhou, China, in 1991, the M.S. degree in fundamental mathematics from Anhui University, Wuhu, China, in 1998, and the Ph.D. degree in control theory and control engineering from the Beijing University of Aeronautics and Astronautics, Beijing, China, in 2001.

His current research interests include the fields of networked control systems, robust control and signal processing, active disturbance rejection control, and flight control.



Antonios Tsourdos (Member, IEEE) received the Ph.D. degree in nonlinear robust autopilot design and analysis from Cranfield University, Cranfield, U.K., in 1999.

He joined the Cranfield University in 1999 and currently serve as the Director of Research - Aerospace, Transport and Manufacturing in 2015. He conducts basic and applied research in the fields of guidance, control, and navigation for single and multiple unmanned autonomous vehicles as well as research on cyber-physical

systems.